VŠB TECHNICKÁ UNIVERZITA OSTRAVA | FAKULTA ELEKTROTECHNIKY A INFORMATIKY

Komponenta výukového serveru TI - P-úplné problémy

Educational Server Component for Theoretical Computer Science: P-complete Problems

Tomáš Kirnig

Bakalářská práce

Vedoucí práce: Ing. Martin Kot, Ph.D.

Ostrava, 2025

Č.j.VSB/25/116933
vsboee25037FA8

Zadání bakalářské práce

Student: **Tomáš Kirnig**

Studijní program: B0613A140014 Informatika

Téma: Komponenta výukového serveru TI - P-úplné problémy
Component of Teaching Server for Theoretical Computer Science - P-complete problems

Jazyk vypracování: čeština

Zásady pro vypracování:

V rámci diplomových a bakalářských prací vzniká výukový server pro předměty teoretické informatiky. Jedná se o sadu dynamických webových stránek umožňujících studentům pochopení různých typů úloh a problémů. Na rozdíl od běžných výukových textů s pevně daným počtem ukázkových příkladů umí tyto stránky generovat libovolně mnoho ukázek na základě vstupů od uživatele. Cílem této konkrétní bakalářské práce je vytvořit komponentu pro pomoc s výukou tzv. P-úplných problémů.

Vytvořte dynamické webové stránky umožňující uživateli následující:
1. Simulovat výpočet řešení problému Monotone Circuit Value Problem (MCVP) a alespoň 2 dalších P-úplných problémů.
2. Vstupy těchto algoritmů bude moci uživatel zadávat třemi způsoby:
a) Vhodným, uživatelsky přívětivým, způsobem ručně.
b) Nechat si vstup vygenerovat zcela náhodně podle nastavených parametrů.
c) Vybrat z předpřipravené sady vhodně zvolených vstupů.
3. Bude možné si zobrazit převod instance problému MCVP na ty dva zvolené P-úplné problémy. Přitom:
a) Instanci MCVP pro převod bude možné zadat kterýmkoliv z výše uvedených způsobů.
b) Převod si bude moci uživatel krokovat se zobrazením slovního vysvětlení jednotlivých kroků.
c) Na převodem vytvořenou instanci bude opět možné použít výše požadovanou simulaci výpočtu řešení.

Seznam doporučené odborné literatury:

[1] Miyano, S., Shiraishi, S., Shoudai, T.: "A List of P-Complete Problems", Kyushu University, RIFIS-TR-CS-17, December 1990, dostupné z URL: https://catalog.lib.kyushu-u.ac.jp/opac_download_md/3123/rifis-tr-17.pdf
[2] Sawa, Z.: "Teoretická informatika", podklady pro přednášky, VŠB - Technická univerzita Ostrava, dostupné z URL: https://www.cs.vsb.cz/sawa/ti/slides/ti-slides-03.pdf
[3] Papadimitriou, C.: Computational Complexity, Addison Wesley, 1993
[4] Arora, S., Barak, B.: Computational Complexity: A Modern Approach, Cambridge University Press, 2009

Formální náležitosti a rozsah bakalářské práce stanoví pokyny pro vypracování zveřejněné na webových stránkách fakulty.

Vedoucí bakalářské práce: **Ing. Martin Kot, Ph.D.**

Datum zadání: 01.09.2024

Datum odevzdání: 30.04.2026

Garant studijního programu: Ing. Tomáš Fabián, Ph.D.

V IS EDISON zadáno: 18.08.2025 09:42:23

Abstrakt

Tato bakalářská práce se zabývá vývojem výukové webové aplikace pro demonstraci P-úplných problémů. Cílem je usnadnit studentům pochopení a procvičování těchto problémů. Aplikace se zaměřuje na tři P-úplné problémy: *Monotone Circuit Value Problem* (MCVP), *problém neprázdnosti jazyka bezkontextové gramatiky* a *kombinatorické hry na grafu*. Práce zahrnuje implementaci modulů pro interaktivní zadávání vstupů (ručně, náhodnou generací nebo výběrem z připravené sady) a simulaci jejich řešení. Uživatel může také sledovat krokovaný převod instance MCVP na další zmíněné problémy a následně jejich řešení.

Klíčová slova

P-úplnost, Monotone Circuit Value Problem, logaritmické redukce, kombinatorické hry na grafu, problém neprázdnosti jazyka bezkontextové gramatiky, polynomiální redukce, vizualizace algoritmů, interaktivní výuková webová aplikace

Abstract

This bachelor's thesis focuses on the development of a teaching-oriented web application for illustrating P-complete problems. The main goal is to facilitate students' understanding and practice of such tasks. The application focuses on the *Monotone Circuit Value Problem* (MCVP), *Context-Free Grammar Non-emptiness Problem*, and *Combinatorial Games on Graphs*. The project implements modules for interactive input of problem instances (manually, via random generation, or by selecting from a pre-defined set) and provides a simulation of their solutions. Users can also observe a step-by-step reduction from an MCVP instance to the other mentioned problems and subsequently explore how those are solved. The resulting application demonstrates key concepts of theoretical computer science, including the notion of P-completeness, and provides a flexible basis for educational use.

Keywords

P-completeness, Monotone Circuit Value Problem, logspace reductions, combinatorial games on graphs, context-free grammar non-emptiness problem, polynomial-time reductions, algorithm visualization, interactive educational web application

Obsah

Seznam použitých symbolů a zkratek **7**

Seznam obrázků **8**

1 Úvod **9**

2 Teoretický základ **11**
2.1 Základní pojmy a definice 11
2.2 Monotone Circuit Value Problem 12
2.3 Kombinatorické hry na grafu 12
2.4 Neprázdnost jazyka bezkontextové gramatiky 13
2.5 Principy převodů mezi problémy 13

3 Analýza a návrh **16**
3.1 Volba technologií 16
3.2 Architektura a struktura kódu 16
3.3 Návrh datových struktur 17
3.4 Návrh uživatelského rozhraní 18

4 Implementace **20**
4.1 Monotone Circuit Value Problem 20
4.2 Kombinatorická hra 26
4.3 Bezkontextové gramatiky 31
4.4 Implementace převodů mezi problémy 37

5 Závěr **42**
5.1 Dosažené výsledky 42
5.2 Vzdělávací přínos 43
5.3 Možnosti dalšího rozvoje 43

Literatura **44**

Přílohy **45**

A Obsah přiloženého archivu **46**

B Technická příručka pro spuštění a nasazení **47**

B.1 Systémové požadavky 47

B.2 Instalace a lokální spuštění 47

B.3 Produkční nasazení 48

Seznam použitých zkratek a symbolů

MCVP	–	Monotone Circuit Value Problem
AND	–	Logický součin – hradlo typu AND
OR	–	Logický součet – hradlo typu OR
P	–	Třída problémů řešitelných v polynomiálním čase
NC	–	Nick's Class – třída efektivně paralelizovatelných problémů
BFS	–	Breadth First Search – Algoritmus průchodu do šířky
CFG	–	Context-Free Grammar – Bezkontextová gramatika
DAG	–	Directed Acyclic Graph – Orientovaný acyklický graf
DFS	–	Depth First Search – Algoritmus průchodu do hloubky
DOM	–	Document Object Model – Objektový model dokumentu
ID	–	Identifier – Identifikátor
JSON	–	JavaScript Object Notation – Objektový zápis JavaScriptu
LS	–	Levá strana
PS	–	Pravá strana
SPA	–	Single Page Application – Jednostránková webová aplikace

Seznam obrázků

3.1 Třídní diagram pro hlavní třídy MCVP 17
3.2 Třídní diagram pro hlavní třídy kombinatorické hry 18
3.3 Třídní diagram pro hlavní třídu bezkontextové gramatiky 18
3.4 Ovládací panel s navigací a výběrem vstupu 19
3.5 Interaktivní plátno pro vizualizaci a editaci grafů 19
3.6 Informační panel zobrazující výsledek analýzy 19

4.1 Výběr způsobu zadání vstupu pro MCVP 20
4.2 Interaktivní editor pro tvorbu a úpravu monotónních obvodů 24
4.3 Ukázka JSON formátu pro uložení MCVP obvodu 25
4.4 Vizualizace MCVP obvodu pomocí TreeRenderCanvas 26
4.5 Ukázka předpřipravených herních grafů ve formátu JSON 28
4.6 Ovládací prvky pro import a export herních grafů 28
4.7 Kroková analýza kombinatorické hry 30
4.8 Rozhraní pro nastavení parametrů generování bezkontextových gramatik 33
4.9 Ukázka formátu JSON pro uložení bezkontextové gramatiky 35
4.10 Vizualizace derivačního stromu pro neprázdnou gramatiku 36
4.11 Kroková analýza bezkontextové gramatiky 37
4.12 Ukázka převodu MCVP obvodu na kombinatorickou hru 39
4.13 Ukázka převodu MCVP obvodu na bezkontextovou gramatiku 41

Kapitola 1

Úvod

Teoretická informatika poskytuje způsob, jak zkoumat, porozumět a optimalizovat možnosti a limity výpočetních systémů. Hlavním bodem tohoto zkoumání je teorie složitosti, která klasifikuje problémy na základě zdrojů potřebných pro jejich vyřešení, jako je čas a paměť. Jednou z nejvýznamnějších složitostních tříd je třída P, zahrnující problémy řešitelné v polynomiálním čase na deterministickém Turingově stroji [10] – tedy takové, jejichž časovou složitost lze vyjádřit jako $O(n^k)$ [11] pro nějakou konstantu k, kde n je velikost vstupu. Přestože jsou problémy v této třídě obvykle pokládány za „efektivně řešitelné“, projevují se mezi nimi významné odlišnosti, zejména pokud začneme řešit jejich paralelizovatelnost.

V tomto kontextu hraje klíčovou roli podmnožina *P-úplných* problémů (*P-complete* problems). P-úplné problémy představují výpočetně nejnáročnější úlohy v rámci třídy P. Jsou to problémy, na které lze s logaritmickou paměťovou složitostí převést jakýkoliv jiný problém z třídy P [7, 1]. Hlavní problém u této podmnožiny spočívá v předpokladu, že P-úplné problémy pravděpodobně nelze efektivně paralelizovat. To znamená, že tyto úlohy nespadají do třídy NC, která obsahuje problémy řešitelné v polylogaritmickém čase $O(\log^k n)$ pomocí paralelního výpočtu s polynomiálně mnoha procesory [8]. Z toho vyplývá, že na rozdíl od NC problémů řešení P-úplných problémů vyžaduje sekvenční zpracování. Studium P-úplnosti nám tak pomáhá vymezit hranici mezi paralelizovatelným a čistě sekvenčním výpočtem.

Tato bakalářská práce představuje interaktivní ukázku konceptu P-úplnosti prostřednictvím webové aplikace. Cílem je vytvořit nástroj, který umožní uživatelům vizualizovat a pochopit výpočet a převod mezi zvolenými P-úplnými problémy.

Jako hlavní problém byl zvolen *Monotone Circuit Value Problem* (MCVP), ve kterém se vyhodnocuje logický obvod tvořený pouze hradly typu AND a OR [8]. Pro demonstraci univerzálnosti konceptu P-úplnosti aplikace implementuje také dva další problémy:

- **Kombinatorické hry na grafu:** Problém určení existence vítězné strategie v deterministické hře dvou hráčů [8].

- **Problém neprázdnosti jazyka bezkontextové gramatiky:** Rozhoduje, zda daná gramatika generuje neprázdný jazyk [8].

Hlavním přínosem vytvořené aplikace je možnost vizualizace nejen samotného řešení těchto úloh, ale především *převodů* (redukcí) mezi nimi. Uživatel může sledovat krokovanou transformaci instance MCVP na instanci hry nebo gramatiky, což názorně ilustruje princip polynomiálních redukcí a vzájemnou převoditelnost P-úplných problémů [8].

Výsledkem práce je webová aplikace pro výuku P-úplnosti, která podporuje manuální zadávání instancí, jejich náhodné generování i práci s předpřipravenými sadami.

Práce je členěna do několika částí. Po úvodu následuje kapitola věnovaná teoretickému základu, definicím klíčových pojmů a principům redukcí. Další část se zabývá analýzou a návrhem aplikace, volbou technologií a popisem datových struktur. Následuje kapitola věnovaná detailnímu popisu implementace všech tří problémů a jejich vzájemných převodů. Závěr práce shrnuje dosažené výsledky a navrhuje možnosti dalšího rozšíření.

Kapitola 2

Teoretický základ

Tato kapitola zavádí základní pojmy z teorie složitosti a popisuje tři P-úplné problémy: Monotone Circuit Value Problem, kombinatorické hry na grafu a problém neprázdnosti jazyka bezkontextové gramatiky.

2.1 Základní pojmy a definice

Nejprve jsou zavedeny klíčové pojmy z teorie složitosti, o které se tato práce opírá.

Definice 1 (Třída PTIME (P) [10]) *Třída **PTIME** (běžně značená také jako **P**) obsahuje všechny rozhodovací problémy, které jsou řešitelné na deterministickém Turingově stroji v čase $O(n^k)$, kde n je velikost vstupu a k je nezáporná konstanta.*

Definice 2 (Logaritmická redukce [8]) *Nechť A a B jsou jazyky (problémy). Řekneme, že A je **převeditelný v logaritmickém prostoru** na B (značíme $A \leq_L B$), jestliže existuje funkce f vyčíslitelná Turingovým strojem s logaritmickou paměťovou složitostí taková, že pro každé slovo w platí:*

$$w \in A \iff f(w) \in B$$

(slovo w patří do jazyka A právě tehdy, když jeho obraz $f(w)$ patří do jazyka B).

Definice 3 (P-úplnost [8, 7]) *Problém A se nazývá **P-úplný**, jestliže platí dvě podmínky:*

1. *$A \in P$ (problém patří do třídy P).*
2. *Pro každý problém $B \in P$ platí $B \leq_L A$ (každý problém z třídy P lze na problém A převést pomocí redukce v logaritmickém prostoru).*

2.2 Monotone Circuit Value Problem

Monotone Circuit Value Problem je základní problém v teorii složitosti, který patří mezi P-úplné problémy [8]. Jeho definice je následující:

- **Vstup:** Booleovský obvod bez negací, tvořený pouze hradly AND ($\wedge$) a OR ($\vee$), společně se vstupními hodnotami (pouze hodnoty 0 nebo 1 - false nebo true).
- **Výstup:** Hodnota výstupního uzlu obvodu (výstupního hradla).

Obvod lze reprezentovat jako orientovaný acyklický graf (DAG), kde uzly představují buď vstupní proměnné (listy) nebo logická hradla (vnitřní uzly). Hrany reprezentují tok logických hodnot – přenášejí výsledky vyhodnocení z jednoho uzlu jako vstupy do uzlů následujících. V monotónním obvodu se nevyskytují hradla NOT (negace), což zajišťuje, že funkce reprezentovaná obvodem je monotónní – zvýšení hodnoty libovolného vstupu nikdy nesníží hodnotu výstupu [8].

2.2.1 P-úplnost MCVP

MCVP je P-úplný problém [8, 6]. Vyhodnocení monotónního obvodu lze provést v polynomiálním čase postupným vyhodnocováním uzlů od vstupů směrem k výstupu. MCVP je sice řešitelný v polynomiálním čase, ale jeho P-úplnost znamená, že efektivní paralelizace je pravděpodobně nemožná – vyhodnocení obvodu musí probíhat sekvenčně od vstupů k výstupu.

Pro P-úplnost MCVP je zásadní teoretický model, ve kterém je vstupní obvod reprezentován jako DAG, nikoliv jako strom. Tentýž logický výraz může být jako DAG výrazně menší než odpovídající strom, protože v DAGu se stejné podformule vyskytují pouze jednou, zatímco ve stromě tolikrát, kolikrát je jejich hodnota dále využívána. Velikost vstupu n je tedy u DAGu menší a při složitosti $O(n^k)$ je k dispozici méně kroků na provedení výpočtu. Dosáhnout polynomiální složitosti je tak na DAGu těžší než na stromě – problém vyhodnocení na stromě je sice v P, ale předpokládá se, že není P-těžký (a tedy ani P-úplný).

2.3 Kombinatorické hry na grafu

Kombinatorická hra dvou hráčů na orientovaném grafu je další příklad P-úplného problému [8]:

- **Vstup:** Orientovaný graf, kde každý uzel představuje herní pozici přiřazenou Hráči 1 nebo Hráči 2, hrany reprezentují možné tahy a jeden uzel je označen jako startovní pozice.
- **Výstup:** Rozhodnutí, zda Hráč 1 má výherní strategii ze startovní pozice.

Hráči se střídají v tazích podle toho, komu je přiřazen aktuální uzel. Hra končí, když hráč na tahu nemá žádný možný tah – tento hráč pak prohrává. Výherní strategie pro Hráče 1 znamená, že existuje způsob volit tahy tak, aby Hráč 1 vyhrál bez ohledu na to, jak hraje Hráč 2 [8].

2.3.1 P-úplnost kombinatorických her

Tento problém je P-úplný [8, 6]. Určení výherní strategie lze provést v polynomiálním čase pomocí tzv. retrográdní analýzy [8], která zpětně vyhodnocuje pozice od koncových uzlů. P-úplnost problému implikuje obtížnost paralelizace: výherní strategie nelze určit efektivně paralelně, protože analýza pozic na sebe vzájemně závisí.

2.4 Neprázdnost jazyka bezkontextové gramatiky

Problém neprázdnosti jazyka bezkontextové gramatiky (CFG Non-emptiness Problem) je dalším příkladem P-úplného problému [8]. Bezkontextová gramatika generuje jazyk pomocí přepisovacích pravidel.

Formálně je gramatika definována jako čtveřice $G = (N, \Sigma, P, S)$ [9], kde:

- N je konečná množina neterminálních symbolů (neterminálů).
- Σ je konečná množina terminálních symbolů (terminálů), disjunktní s N.
- P je konečná množina přepisovacích pravidel tvaru $A \to \alpha$, kde $A \in N$ a $\alpha \in (N \cup \Sigma)^*$.
- $S \in N$ je počáteční symbol (start symbol).

Problém spočívá v tom, zda daná gramatika generuje alespoň jedno slovo složené pouze z terminálních symbolů (včetně prázdného řetězce - ε).

2.4.1 P-úplnost problému neprázdnosti

Problém neprázdnosti jazyka bezkontextové gramatiky je P-úplný [8, 6]. Tento problém lze vyřešit v polynomiálním čase pomocí algoritmu iterativního označování produktivních neterminálů. Podobně jako u předchozích problémů, i zde P-úplnost vyplývá ze sekvenčních závislostí: produktivita neterminálu závisí na produktivitě jiných neterminálů, což brání paralelnímu zpracování.

2.5 Principy převodů mezi problémy

P-úplné problémy jsou vzájemně převoditelné, což dokazuje jejich výpočetní rovnocennost. Následující převody ilustrují, jak lze MCVP převést na kombinatorickou hru a na problém neprázdnosti jazyka bezkontextové gramatiky.

2.5.1 Převod z MCVP na kombinatorickou hru

Konstrukce převodu

Pro zadaný monotónní obvod vytvoříme instanci hry takto [8]:

- Z každého hradla OR a každé proměnné s hodnotou 0 se vytvoří pozice, kde je na tahu Hráč 1.
- Z každého hradla AND a každé proměnné s hodnotou 1 se vytvoří pozice, kde je na tahu Hráč 2.
- Startovní pozice odpovídá výstupnímu (kořenovému) uzlu obvodu.
- Z pozice odpovídající každému hradlu vedou orientované hrany do pozic odpovídajících všem potomkům tohoto hradla v obvodu.
- Pozice odpovídající vstupním proměnným jsou koncové (bez odchozích hran).

Zdůvodnění korektnosti

Než přejdeme k detailům, je vhodné zdůraznit princip celé konstrukce. Vyhodnocení MCVP obvodu probíhá od vstupů směrem k výstupu, zatímco kombinatorická hra se hraje opačně: ze startovní pozice odpovídající výstupu se postupuje po hranách k listům. Převod je navržen tak, aby Hráč 1 měl výherní strategii právě tehdy, když se MCVP obvod vyhodnotí na 1. Rozhodování v herních pozicích proto odpovídá podmínkám, za nichž jednotlivá hradla získají hodnotu 1.

S tímto pohledem korektnost plyne z významu jednotlivých typů pozic:

- **OR pozice (Hráč 1):** Hráč 1 volí pokračování. K výhře mu stačí možnost jedné výherní volby, což odpovídá tomu, že hradlo OR má hodnotu 1, když má alespoň jeden vstup hodnotu 1.
- **AND pozice (Hráč 2):** Hráč 2 volí pokračování a může vybrat případný vstup s hodnotou 0. Hráč 1 tedy vyhraje jen tehdy, když jsou výherní všechna pokračování, což odpovídá tomu, že hradlo AND má hodnotu 1 pouze při samých jedničkách na vstupech.
- **Listy:** Proměnná s hodnotou 1 je pozice Hráče 2 bez tahu (Hráč 2 prohrává), zatímco proměnná s hodnotou 0 je pozice Hráče 1 bez tahu (Hráč 1 prohrává).

Proto platí ekvivalence: Hráč 1 má výherní strategii ze startovní pozice právě tehdy, když výstup obvodu má hodnotu 1.

2.5.1.0.1 Paměťová složitost převodu. Převod lze provést v logaritmickém prostoru. Vstupní obvod ani zapisovaný výstupní graf hry se do pracovní paměti nepočítají; v paměti se drží pouze několik čítačů a pomocných proměnných pro iteraci přes uzly a hrany. Hodnoty těchto čítačů jsou nejvýše velikosti počtu uzlů a hran obvodu, takže jejich reprezentace vyžaduje $O(\log|V|+\log|E|) = O(\log n)$ bitů, kde n je velikost vstupního obvodu.

2.5.2 Převod z MCVP na bezkontextovou gramatiku

Konstrukce převodu

Převod je proveden v několika krocích [8]:

- Každému uzlu operace v obvodu je přiřazen neterminál gramatiky.
- Neterminál odpovídající výstupnímu uzlu obvodu je zvolen jako počáteční symbol S.
- Pokud neterminál X reprezentuje hradlo AND s potomky reprezentovanými neterminály A a B, je přidáno pravidlo $X \to AB$.
- Pokud neterminál X reprezentuje hradlo OR s potomky reprezentovanými neterminály A a B, jsou přidána pravidla $X \to A$ a $X \to B$.
- Pokud uzel reprezentuje proměnnou s hodnotou 1, je převeden na terminální symbol a je použito pravidlo $X \to a$, případně (s pravděpodobností 15%) $X \to \varepsilon$.
- Pokud uzel reprezentuje proměnnou s hodnotou 0, je reprezentován neterminální smyčkou $X \to X$.

Zdůvodnění korektnosti

Korektnost je založena na tvrzení, že neterminál je produktivní právě tehdy, když odpovídající uzel obvodu má hodnotu 1:

- **Proměnná 1:** Z pravidla $X \to a$, včetně případu $X \to \varepsilon$, plyne, že X je produktivní.
- **Proměnná 0:** Z pravidla $X \to X$ nelze odvodit terminální řetězec, takže X není produktivní.
- **OR hradlo:** Neterminál X je produktivní právě tehdy, když je produktivní alespoň jeden potomek (existuje pravidlo $X \to A$ nebo $X \to B$ vedoucí k terminálnímu řetězci).
- **AND hradlo:** Neterminál X je produktivní právě tehdy, když jsou produktivní oba potomci, protože pravidlo $X \to AB$ vyžaduje odvodit terminální řetězec z A i z B.

Z toho přímo plyne, že počáteční symbol je produktivní právě tehdy, když má výstupní uzel obvodu hodnotu 1. Jazyk výsledné gramatiky je tedy neprázdný právě tehdy, když se obvod vyhodnotí na 1.

2.5.2.0.1 Paměťová složitost převodu. Také tento převod je realizovatelný v logaritmickém prostoru. Vstupní obvod a zapisovaná výstupní pravidla gramatiky se nepočítají do pracovní paměti; algoritmus potřebuje jen čítače pro procházení uzlů a hran a krátkodobé uložení právě zpracovávané lokální informace. Proto opět stačí $O(\log |V| + \log |E|) = O(\log n)$ bitů pracovní paměti.

Kapitola 3

Analýza a návrh

Tato kapitola popisuje zvolené technologie a architekturu aplikace. Dále představuje datové struktury pro reprezentaci jednotlivých problémů a principy návrhu uživatelského rozhraní.

3.1 Volba technologií

Aplikace byla implementována jako webová, protože nevyžaduje instalaci a běží v libovolném moderním prohlížeči. Webové řešení také umožňuje snadnou údržbu a okamžitou distribuci aktualizací.

3.1.1 React a Vite

Pro vývoj byla použita knihovna *React* [5]. React umožňuje rozdělit uživatelské rozhraní na znovupoužitelné komponenty, což usnadnilo vývoj modulů pro jednotlivé problémy. Díky reaktivitě se rozhraní automaticky synchronizuje se stavem dat bez nutnosti ruční manipulace s DOM.

Pro sestavení aplikace byl použit nástroj *Vite* [12]. Vite nabízí rychlé vývojové prostředí s okamžitou odezvou při úpravách kódu a optimalizuje soubory pro produkční verzi.

3.2 Architektura a struktura kódu

Aplikace je navržena jako *Single Page Application* (SPA) – veškerá logika a navigace probíhá v rámci jedné stránky, podobně jako u desktopové aplikace. Kořenová komponenta `App.jsx` spravuje globální stav a přepínání mezi moduly.

Kód je strukturován tak, aby logika byla oddělena od vizualizace. Každý modul (MCVP, hry, gramatiky) disponuje vlastní adresářovou strukturou:

- **Hlavní komponenta:** Funguje jako kontejner, který spravuje lokální stav a propojuje vstupní data s algoritmy a vizualizací.

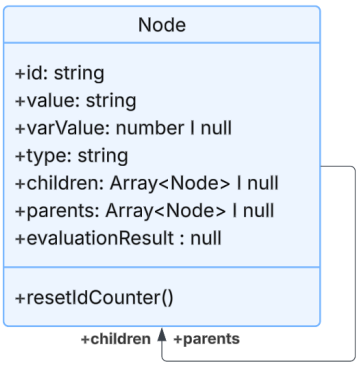
- **Adresář Utils:** Zde jsou soustředěny čisté algoritmy – parsery pro zpracování vstupu, generátory náhodných instancí a evaluátory pro výpočet řešení.
- **Vizualizační komponenty:** Starají se o vykreslení grafů a stromů pomocí knihovny *react-force-graph*.

3.3 Návrh datových struktur

Pro práci s P-úplnými problémy bylo nutné navrhnout jejich vnitřní reprezentaci v paměti.

3.3.1 Reprezentace obvodu MCVP

Pro modelování logického obvodu byl využit orientovaný acyklický graf (DAG). Každý uzel obvodu je reprezentován instancí třídy `Node`. Tato třída, jak ukazuje třídní diagram na obrázku 3.1, uchovává informaci o typu operace (AND, OR) nebo hodnotě proměnné. Seznam odkazů na potomky umožňuje průchod grafem při vyhodnocování.



Obrázek 3.1: Třídní diagram pro hlavní třídy MCVP

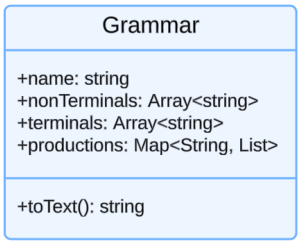
3.3.2 Reprezentace herního grafu

Na rozdíl od MCVP může herní graf obsahovat cykly, proto byly navrženy třídy `GamePosition` a `GameGraph` (viz obrázek 3.2). Třída `GamePosition` reprezentuje jednotlivou pozici a uchovává informaci o hráči na tahu spolu se seznamy předchůdců a následovníků, což je nezbytné pro retrográdní analýzu.

3.3.3 Reprezentace bezkontextové gramatiky

Třída `Grammar` (obrázek 3.3) spravuje množiny symbolů a seznamy pravidel. Každé pravidlo propojuje levou stranu (neterminál) s polem symbolů na pravé straně. Tato struktura umožňuje snadno identifikovat produktivní neterminály.

Obrázek 3.2: Třídní diagram pro hlavní třídy kombinatorické hry



Obrázek 3.3: Třídní diagram pro hlavní třídu bezkontextové gramatiky

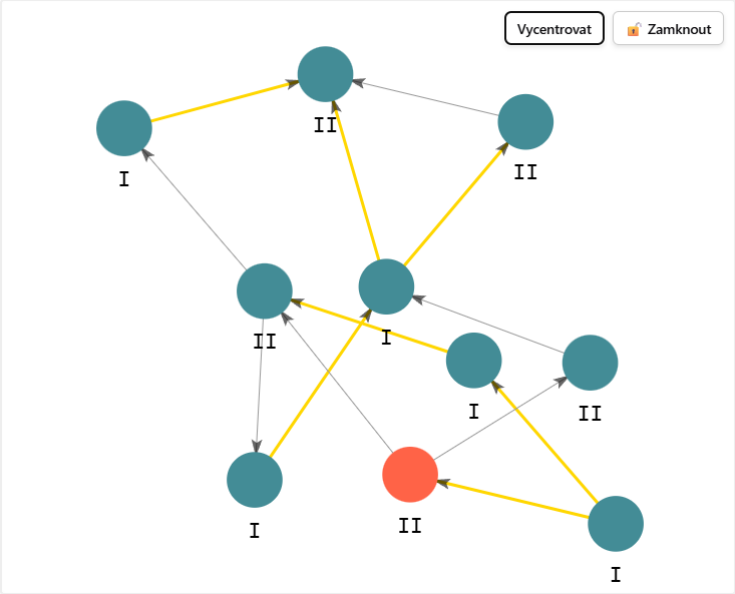
3.4 Návrh uživatelského rozhraní

Rozhraní je navrženo tak, aby student nebyl zahlcen informacemi a mohl se soustředit na podstatu problému. Rozložení stránky bylo sjednoceno napříč všemi moduly:

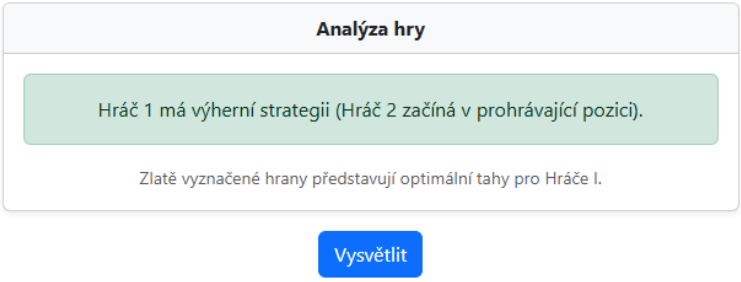
- **Ovládací panel:** Horní část obrazovky, kde uživatel volí způsob zadání vstupu a spouští simulace (viz obrázek 3.4).
- **Interaktivní plátno:** Hlavní plocha pro vizualizaci grafů, kde uživatel může s prvky přímo manipulovat (viz obrázek 3.5).
- **Informační panely:** Modální okna nebo postranní panely zobrazující detaily o probíhajícím výpočtu nebo krokovém převodu (viz obrázek 3.6).

Pro vizualizaci byla použita knihovna *react-force-graph* [2], která k výpočtu fyzikální simulace pro automatické rozmísťování uzlů a hran využívá engine *D3.js* [4]. Stylování je realizováno frameworkem *Bootstrap 5* [3], který zajišťuje jednotný vzhled.

Obrázek 3.4: Ovládací panel s navigací a výběrem vstupu



Obrázek 3.5: Interaktivní plátno pro vizualizaci a editaci grafů



Obrázek 3.6: Informační panel zobrazující výsledek analýzy

Kapitola 4

Implementace

Tato kapitola detailně popisuje implementaci jednotlivých modulů aplikace. Zaměřuje se na technické řešení algoritmů pro parsování, vyhodnocování a generování instancí P-úplných problémů.

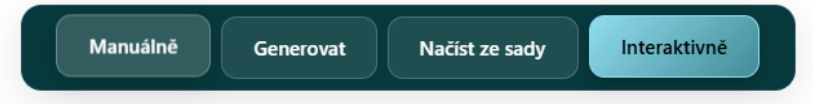
4.1 Monotone Circuit Value Problem

Modul pro MCVP tvoří jádro aplikace a slouží jako výchozí bod pro převody na ostatní problémy.

4.1.1 Formát vstupu

Aplikace nabízí čtyři způsoby zadávání monotónního obvodu (viz obrázek 4.1):

- **Textový zápis:** Zadání pomocí logického výrazu, který je následně parsován (viz sekce 4.1.2).
- **Interaktivní editace:** Vytváření obvodu přímo na plátně pomocí grafického editoru (viz sekce 4.1.8).
- **Generování náhodných obvodů:** Automatické vytvoření obvodu dle parametrů (viz sekce 4.1.9).
- **Předpřipravené sady:** Načtení ukázkových obvodů z připravené knihovny (viz sekce 4.1.10).



Obrázek 4.1: Výběr způsobu zadání vstupu pro MCVP

4.1.2 Manuální zadání a parsování výrazů

Textový zápis ve formě logického výrazu je první metodou zadání vstupu. Tento zápis je následně převeden na vnitřní strukturu obvodu.

4.1.3 Gramatika vstupního jazyka

- Operátory `O` (OR) a `A` (AND) reprezentují logický součet a součin.
- Proměnné jsou zapsány ve tvaru `x1[0]` nebo `x2[1]`, kde číslo v hranatých závorkách udává hodnotu proměnné (0 nebo 1).
- Výrazy lze libovolně uzávorkovat pomocí kulatých závorek pro určení priority vyhodnocení.

Příklad platného výrazu: `((x1[1] A x2[0]) O (x3[1] A x4[1]))`

4.1.4 Lexikální a syntaktická analýza

Převod textového výrazu na vnitřní grafovou strukturu obvodu probíhá ve dvou fázích, které jsou implementovány v modulu `Parser.js`:

1. **Tokenizace (lexikální analýza):** Funkce `tokenize()` rozdělí vstupní řetězec na posloupnost tokenů. Každý token je dvojice (typ, hodnota), například `['LPAREN', '(']` nebo `['VARIABLE', 'x1[1]']`. Tokenizér rozpoznává závorky, operátory (A, O) a proměnné pomocí regulárních výrazů.
2. **Parsování (syntaktická analýza):** Třída `Parser` implementuje rekurzivní sestupný parser respektující prioritu operátorů. Výrazy s operátorem OR mají nejnižší prioritu, následují výrazy s operátorem AND a nejvyšší prioritu mají atomické výrazy v závorkách nebo proměnné. Parser vytváří uzly typu `Node` (viz třídní diagram v sekci 3.3.1, obrázek 3.1), které tvoří strukturu grafu reprezentující zadaný obvod.

Parser současně kontroluje výskyty opakovaných proměnných. Pokud se stejná proměnná v jednom vstupním výrazu objeví vícekrát, používá se v interní reprezentaci již vytvořený uzel a je vyžadována stejná hodnota, jak byla přiřazena v prvním výskytu. Při konfliktu (například `x1[0]` a `x1[1]`) je vstup vyhodnocen jako neplatný a uživateli je vrácena chybová hláška. Implementace v této fázi obecně neslučuje shodné podvýrazy (podstromy), sdílení uzlů se týká pouze opakovaných proměnných.

Výsledkem parsování je orientovaná struktura, kde vstupní uzly jsou proměnné s přiřazenými hodnotami a vnitřní uzly reprezentují logické operace.

4.1.5 Vyhodnocení obvodu

Vyhodnocení monotónního obvodu probíhá rekurzivním průchodem grafu, který je implementován v modulu `EvaluateCircuit.js`.

4.1.6 Algoritmus vyhodnocení

Funkce `evaluateCircuitWithSteps()` používá algoritmus průchodu do hloubky (DFS) s memoizací. Pro maximální využití vypočítaných dat funkce vrací jak výsledek vyhodnocení, tak i sekvenci všech mezivýpočtů, které jsou následovně použity pro krokovou vizualizaci. Průběh algoritmu:

1. **Listy (proměnné):** Pro uzly typu `variable` funkce vrací přímo přiřazenou hodnotu (0 nebo 1).
2. **Vnitřní uzly (operace):** Pro uzly reprezentující logické operace funkce nejdříve vyhodnotí všechny potomky. Pokud je uzel typu AND, výsledek je 1 právě tehdy, když všichni potomci mají hodnotu 1. Pokud je uzel typu OR, výsledek je 1 pouze tehdy, když alespoň jeden potomek má hodnotu 1.
3. **Zaznamenávání kroků:** Po vyhodnocení každého vnitřního uzlu se do pole kroků uloží záznam obsahující vyhodnocovaný uzel, hodnoty jeho potomků a výslednou hodnotu - samotný výsledek uzlu se zároveň uloží do memoizační mapy (cache).
4. **Memoizace:** Vyhodnocené hodnoty uzlů jsou uloženy do cache (slovníku), aby nedocházelo k opakovanému výpočtu stejných uzlů.
5. **Detekce cyklů:** Při vstupu do uzlu se uzel dočasně vloží do množiny `visiting`. Pokud algoritmus během průchodu narazí na uzel, který se v této množině již nachází, identifikuje cyklus a vyvolá chybu. Po dokončení vyhodnocení se uzel z množiny `visiting` odstraní.

Časová složitost algoritmu je $O(n)$, kde n je počet uzlů v obvodu, protože každý uzel je vyhodnocen právě jednou díky memoizaci. Funkce vrací objekt obsahující samotný výsledek a pole s kroky výpočtu.

4.1.7 Krokové vyhodnocení

Vyhodnocení obvodu je provedeno pouze jednou při načtení, výsledek ve formě výstupní hodnoty obvodu je využit pro běžné zobrazení a zaznamenané kroky jsou předány modální komponentě `StepByStepTree` pro krokovou vizualizaci. Tím se předchází opakovanému průchodu grafem a duplicitnímu výpočtu, což je výhodné zejména u rozsáhlejších obvodů.

Komponenta `StepByStepTree` zobrazuje každý krok graficky – aktuálně vyhodnocovaný uzel je zvýrazněn a pod grafem je zobrazena informace o hodnotách vstupů a výsledku operace. Uživatel tak může sledovat, jak se hodnoty šíří od vstupů k výstupu obvodu a lépe pochopit princip vyhodnocování.

4.1.8 Interaktivní editace obvodu

Kromě možnosti textového zadání umožňuje aplikace vytvářet a upravovat obvody pomocí grafického editoru implementovaného v komponentě `InteractiveMCVPGraph` (viz obrázek 4.2). Uživatel může:

- **Přidávat uzly:** Pomocí tlačítek pod grafem lze vložit nové uzly typu AND, OR nebo vstupní proměnnou s hodnotou 0 nebo 1. Nový uzel se umístí do grafu, ale není propojen s ostatními uzly.
- **Vytvářet hrany:** Označený uzel může:
  - Propojit hranou s dalším uzlem.
  - Odebrat již existující hranu.
  - Nastavit na AND nebo OR operaci.
  - Nastavit na proměnnou s hodnotou 0 nebo 1.
  - Odstranit uzel z grafu.
- **Reorganizovat graf:** Uzly lze přesouvat myší pro lepší vizuální uspořádání obvodu.

Při vytváření hran je v editoru vynuceno binární omezení hradel: uzly operací (AND a OR) mohou mít nejvýše dva potomky.

4.1.9 Generování náhodných obvodů

Aplikace obsahuje generátor náhodných obvodů v modulu `Generator.js`. Uživatel může upravit dva parametry:

- **Počet hradel:** Kolik uzlů s operacemi (hradel AND a OR) bude obvod obsahovat.
- **Počet proměnných:** Kolik uzlů s proměnnými (listů) bude obvod obsahovat.

Generování probíhá v následujících krocích:

1. Před samotným generováním je ověřeno omezení pro binární obvod: počet hradel musí být alespoň počet proměnných mínus jedna (`numGates >= numVariables - 1`).
2. Algoritmus vytvoří zadaný počet proměnných, každou s náhodnou hodnotou 0 nebo 1.
3. Algoritmus dále přidává hradla po jednom a v každém kroku pracuje s aktuální množinou kořenů (uzly bez rodiče). Protože všechna hradla mají mít právě dva vstupy, rozlišuje dva režimy: *slučovací* (hradlo převezme dva různé kořeny) a *rozšiřující* (hradlo převezme jeden kořen a druhý vstup získá z již existujícího nekořenového uzlu).

Obrázek 4.2: Interaktivní editor pro tvorbu a úpravu monotónních obvodů

4. Konkrétní režim se volí podle toho, kolik hradel ještě zbývá do konce. Pokud je potřeba počet kořenů rychleji snížit, použije se slučovací režim; pokud je naopak hradel dostatek, použije se režim rozšiřující. Není-li vhodný nekořenový uzel k dispozici, druhý vstup se doplní dalším kořenem, případně se použije tentýž potomek na obou vstupech.

5. Každé nové hradlo je náhodně zvoleno jako AND nebo OR, má přesně dva vstupy a po vytvoření se stává novým kořenem. Kořeny, které byly použity jako vstupy, tím přestávají být kořeny.

6. Po dokončení generování je ověřeno, že zůstal právě jeden kořenový uzel. Tento uzel představuje výstup obvodu.

Výsledný graf je platný DAG.

4.1.10 Připravené sady obvodů

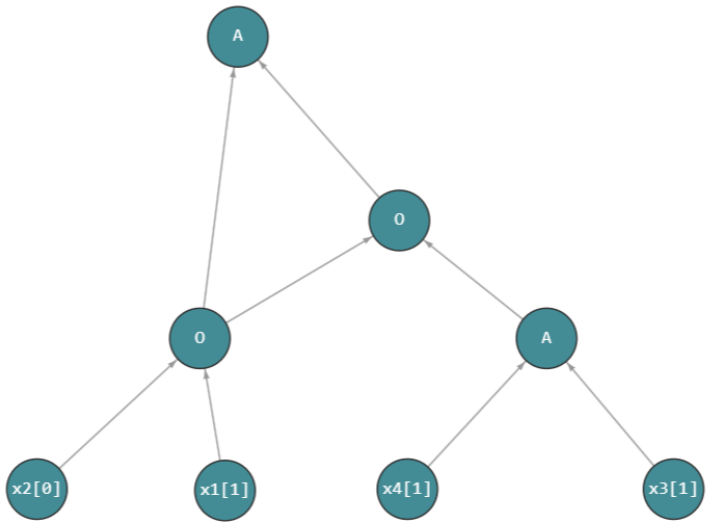
Aplikace obsahuje sadu připravených obvodů, které lze načíst pomocí komponenty `PreparedSetsInput`. Tyto obvody jsou uloženy ve složce `Sady/MCVP` ve formátu JSON, který obsahuje pole `nodes` s uzly a pole `edges` s hranami. Každý uzel má atributy `id`, `type` (AND, OR, variable), a pro proměnné také `value` (0 nebo 1). Hrany jsou reprezentovány jako dvojice `source` a `target`, které odkazují na ID uzlů.

```
1  {
2    "name": "Sada 1 (4x Proměnné, 3x Operace)",
3    "nodes": [
4      { "id": 1, "type": "variable", "value": "x1", "varValue": 0 },
5      { "id": 2, "type": "variable", "value": "x2", "varValue": 1 },
6      { "id": 3, "type": "variable", "value": "x3", "varValue": 1 },
7      { "id": 4, "type": "variable", "value": "x4", "varValue": 0 },
8      { "id": 5, "type": "operation", "value": "O" },
9      { "id": 6, "type": "operation", "value": "A" },
10     { "id": 7, "type": "operation", "value": "A" }
11   ],
12   "edges": [
13     { "source": 5, "target": 1 },
14     { "source": 5, "target": 2 },
15     { "source": 6, "target": 3 },
16     { "source": 6, "target": 4 },
17     { "source": 7, "target": 5 },
18     { "source": 7, "target": 6 }
19   ]
20 }
```

Obrázek 4.3: Ukázka JSON formátu pro uložení MCVP obvodu

4.1.11 Vizualizace obvodu

Vizualizace monotónního obvodu je implementována v komponentě `TreeRenderCanvas`. Obvod je zobrazen jako orientovaný graf s výstupním hradlem nahoře a vstupy dole, hodnoty se tedy šíří směrem zdola nahoru (viz obrázek 4.4).



Obrázek 4.4: Vizualizace MCVP obvodu pomocí TreeRenderCanvas

Uzly jsou automaticky umístěny do vrstev podle jejich vzdálenosti od kořene díky režimu `dagMode="td"` (top-down neboli shora dolů). Rozložení kombinuje hierarchickou strukturu s fyzikálními silami pro optimální vizuální uspořádání grafu.

4.2 Kombinatorická hra

4.2.1 Formát vstupu

Aplikace nabízí tři způsoby zadávání herního grafu:

- **Interaktivní editace:** Uživatel vytváří a upravuje graf pomocí grafického editoru (viz sekce 4.2.2).
- **Generování náhodných her:** Automatické vytvoření náhodného grafu podle zadaných parametrů (viz sekce 4.2.3).
- **Předpřipravené sady:** Načtení ukázkových připravených příkladů (viz sekce 4.2.4).

Všechny formy vstupu využívají komponentu `DisplayGraph` pro vizualizaci herního grafu (viz sekce 4.2.6).

4.2.2 Interaktivní editace grafu

Komponenta `ManualInput` umožňuje vytvářet a upravovat herní grafy pomocí interaktivního grafického editoru. Uživatel může:

- **Přidávat uzly:** Vytvořit novou pozici.
- **Upravovat uzly:** Po označení uzlu lze:
  - Změnit hráče, který je na tahu (Hráč 1 nebo Hráč 2)
  - Odstranit pozici z grafu
  - Nastavit jako počáteční pozici
  - Vytvořit hranu z tohoto uzlu
  - Odstranit hrany k nebo od tohoto uzlu
- **Reorganizovat graf:** Uzly lze přesouvat myší pro lepší vizuální uspořádání.

Aplikace průběžně validuje graf. Pokud není nastavena počáteční pozice, analýza není provedena a uživateli se zobrazí informační zpráva.

4.2.3 Generování náhodných her

Modul `Generator.js` obsahuje funkci `generateGraph()` pro vytváření náhodných herních grafů. Uživatel nastavuje dva parametry:

- **Počet pozic:** Kolik uzlů (herních pozic) bude graf obsahovat.
- **Pravděpodobnost hrany:** Hodnota 0%–100% určující, jak pravděpodobné je vytvoření hrany mezi dvěma uzly.

Algoritmus generování probíhá ve dvou fázích: nejprve vytvoří kostru grafu, aby byly všechny pozice dosažitelné z počáteční pozice, a poté graf náhodně zahustí přidáváním dalších hran, které mohou vytvářet cykly. Postup je následující:

1. **Vytvoření kostry:** Vytvoří se uzly očíslované 0 až $n-1$ s náhodně přiřazeným hráčem. Pro každý uzel $i > 0$ vznikne hrana z náhodného předchozího uzlu do uzlu i. Tím je zaručena dosažitelnost všech uzlů z uzlu 0 (počáteční pozice).
2. **Přidání dalších hran:** Nad vzniklou kostrou se pro každé dva různé uzly posuzuje možnost přidání hrany v obou směrech. Pokud hrana ještě neexistuje, přidá se podle zadané pravděpodobnosti. Tyto hrany mohou vytvářet cykly a zvyšují složitost hry.

Vnitřní reprezentace grafu pomocí tříd `GamePosition` a `GameGraph` je popsána v sekci 3.3.2. Diagram těchto tříd je uveden na obrázku 3.2.

4.2.4 Předpřipravené sady

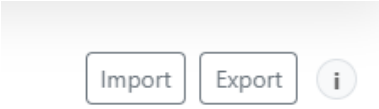
Ve složce `Sady/CombinatorialGame` se nachází předpřipravené herní grafy.

```
1  {
2    "name": "Sada 2 (5 uzlů, 7 hran)",
3    "nodes": [
4      { "id": "1", "player": 1 },
5      { "id": "2", "player": 2 },
6      { "id": "3", "player": 1 },
7      { "id": "4", "player": 2 },
8      { "id": "5", "player": 1 }
9    ],
10   "edges": [
11     { "source": "1", "target": "2" },
12     { "source": "1", "target": "3" },
13     { "source": "2", "target": "4" },
14     { "source": "3", "target": "4" },
15     { "source": "4", "target": "5" },
16     { "source": "2", "target": "3" },
17     { "source": "3", "target": "5" }
18   ],
19   "startingPosition": "3"
20 }
```

Obrázek 4.5: Ukázka předpřipravených herních grafů ve formátu JSON

4.2.5 Ukládání a načítání her

Herní grafy lze exportovat a importovat ve formátu JSON pomocí komponenty `FileTransferControls` (viz obrázek 4.6). Formát obsahuje pole `nodes` s uzly, pole `edges` s hranami a identifikátor `startingPosition` (viz obrázek 4.5).



Obrázek 4.6: Ovládací prvky pro import a export herních grafů

4.2.6 Vizualizace grafu

Vizualizace herního grafu využívá komponentu `DisplayGraph`. Graf zobrazuje:

- **Uzly:** Každý uzel reprezentuje herní pozici s označením hráče na tahu (1 nebo 2).
- **Počáteční pozice:** Označena oranžovou barvou.
- **Hrany:** Možné tahy jsou zobrazeny jako směrované hrany. Hrany patřící do výherní strategie jsou zvýrazněny žlutou barvou.

4.2.7 Algoritmus analýzy a vyhodnocení výherních pozic

Řešení problému kombinatorických her je implementováno v modulu `ComputeWinner.js`. Algoritmus využívá retrográdní analýzu založenou na principech uvedených v sekci 2.3.

Algoritmus určuje, zda je daná pozice pro hráče na tahu výherní, prohrávající, nebo remízová. Vyhodnocení probíhá následovně:

1. **Koncové pozice:** Pozice bez dalších tahů jsou prohrávající pro hráče, který je v nich na tahu.
2. **Zpětné šíření:** Od koncových pozic se postupně propagují výsledky:
   - Je-li z aktuální pozice dostupný alespoň jeden tah do pozice, která je vyhrávající pro hráče, jenž je aktuálně na tahu, je i aktuální pozice vyhrávající.
   - Vedou-li všechny možné tahy do pozic vyhrávajících pro soupeře, je aktuální pozice prohrávající (tj. vyhrávající pro soupeře).
3. **Neurčené pozice:** Pozice, jejichž status nelze retrográdní analýzou označit jako výhru ani prohru, zůstávají označeny jako remíza. Typicky jde o pozice v cyklech (často uvnitř silně souvislé komponenty), ve kterých při optimální hře (oba hráči v každém tahu volí pro sebe nejlepší možný tah) nelze vynutit přechod do jednoznačně prohrávající pozice soupeře. Samotná existence silně souvislé komponenty ale remízu nezaručuje.

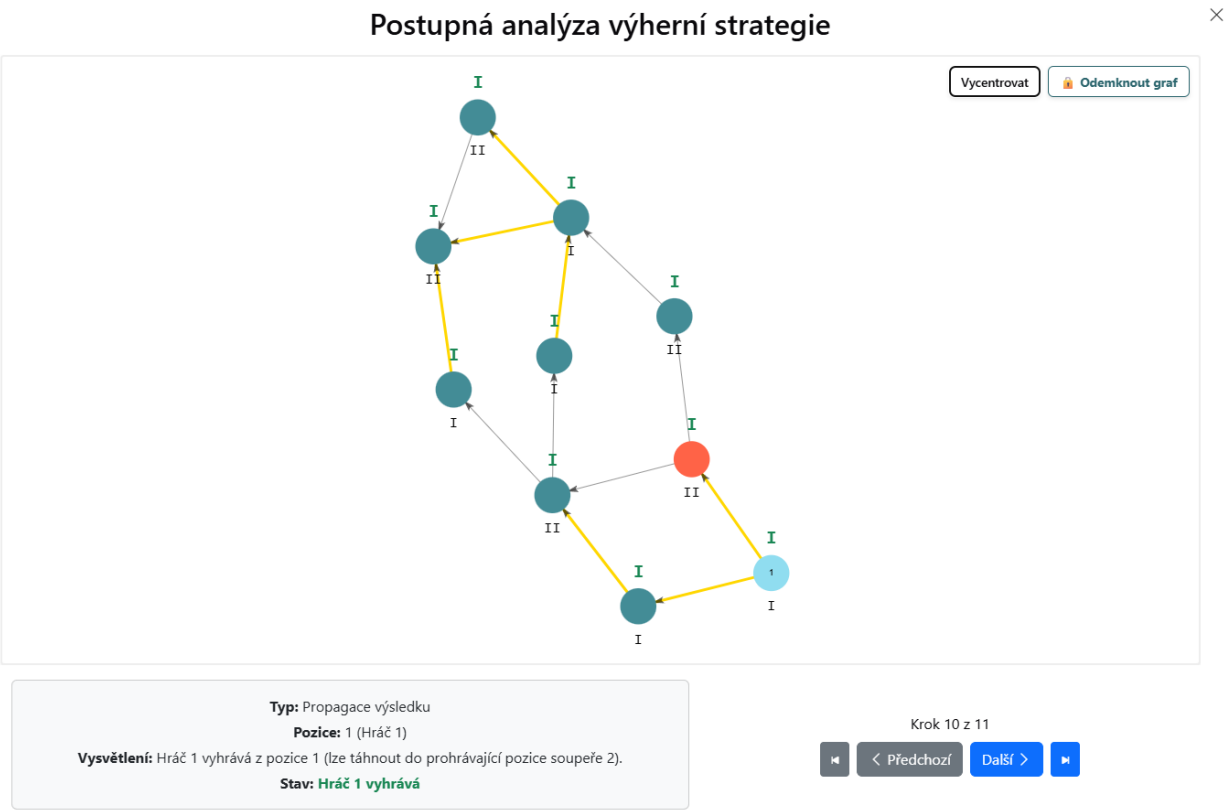
Časová složitost je $O(V + E)$, kde V je počet uzlů a E počet hran.

4.2.8 Krokové vyhodnocení

Součástí analýzy je i funkce `getOptimalMoves()`, která identifikuje hrany patřící do výherní strategie [8]. Hrana z pozice u do pozice v je optimální, pokud obě pozice jsou výherní pro Hráče 1; takové hrany jsou ve vizualizaci zvýrazněny žlutě.

Pro detailnější průchod grafem implementuje komponenta `StepByStepGame` krokovou analýzu (viz obrázek 4.7). Retrográdní analýza je provedena pouze jednou a během tohoto běhu se průběžně ukládají i mezivýpočty (změny stavů uzlů při propagaci výsledků). Uživatel pak může těmito uloženými kroky procházet pomocí navigačních tlačítek. Pro každý krok analýzy je zobrazeno:

- **Aktuálně vyhodnocovaný uzel:** Zvýrazněn přímo v grafu.
- **Vysvětlení kroku:** Popis toho, co se v daném kroku děje (nalezení koncové pozice nebo propagace výsledku).
- **Stav z pohledu Hráče 1:** Všechny zprávy konzistentně uvádějí, zda Hráč 1 z dané pozice vyhrává nebo prohrává.



Obrázek 4.7: Kroková analýza kombinatorické hry

- **Zelené číslo nad uzly:** U vyhodnocených pozic je zobrazen vítězný hráč (1 nebo 2) pro danou pozici.

Po dokončení retrográdní analýzy je přidán závěrečný krok, který zobrazuje celkový výsledek hry – zda Hráč 1 má či nemá výherní strategii z počáteční pozice. Tento finální krok je zvýrazněn žlutým pozadím. Tato funkce pomáhá pochopit, jak algoritmus postupně řeší hru a jak se vypořádává s cykly.

4.3 Bezkontextové gramatiky

4.3.1 Formát vstupu

Aplikace nabízí tři způsoby zadávání bezkontextové gramatiky:

- **Manuální zadání:** Přímé vkládání pravidel gramatiky v textovém formátu (viz sekce 4.3.2).
- **Generování náhodných gramatik:** Automatické vytvoření gramatiky na základě zadaných parametrů (viz sekce 4.3.3).
- **Předpřipravené sady:** Načtení ukázkových gramatik z knihovny příkladů (viz sekce 4.3.4).

Všechny formy vstupu využívají stejnou komponentu pro zobrazení derivačního stromu gramatiky implementovanou v souboru `DerivationTreeVisual.jsx`. Struktura gramatiky je popsána v sekci 3.3.3 a třídní diagram je na obrázku 3.3.

4.3.2 Manuální zadání gramatiky

Komponenta `ManualInput` poskytuje rozhraní ve formě textového okna pro přímé zadávání pravidel gramatiky ve formátu `LS -> PS`. Algoritmus v modulu `GrammarParser.js` automaticky rozpoznává neterminály (řetězce začínající velkým písmenem, včetně českých znaků) a terminály (ostatní řetězce). Pravá strana pravidla se po normalizaci bílých znaků rozdělí podle mezer na jednotlivé tokeny (symboly). To umožňuje používat víceznakové neterminály jako `A1`, `Abc` nebo `A_10`.

Parser akceptuje jak zápis šipky `->` složené ze dvou znaků jednoduše zadatelných z klávesnice, i jako symbol `→`. Obě varianty jsou interně sjednoceny na stejnou reprezentaci pravidla, což zjednodušuje ruční zadávání gramatiky. Parser také validuje, zda jsou všechny použité neterminály (symboly na pravých stranách) definovány alespoň jedním pravidlem (výskytem na levé straně); v opačném případě uživatele upozorní chybovou hláškou.

4.3.3 Generování náhodných gramatik

Modul `GrammarGenerator.js` obsahuje funkci `generateGrammar()` pro vytváření náhodných bezkontextových gramatik na základě vstupních parametrů.

Základní parametry

Základní parametry určují počet neterminálů (1–10), počet terminálů (1–10) a maximální délku pravé strany (1–5).

Pokročilé parametry

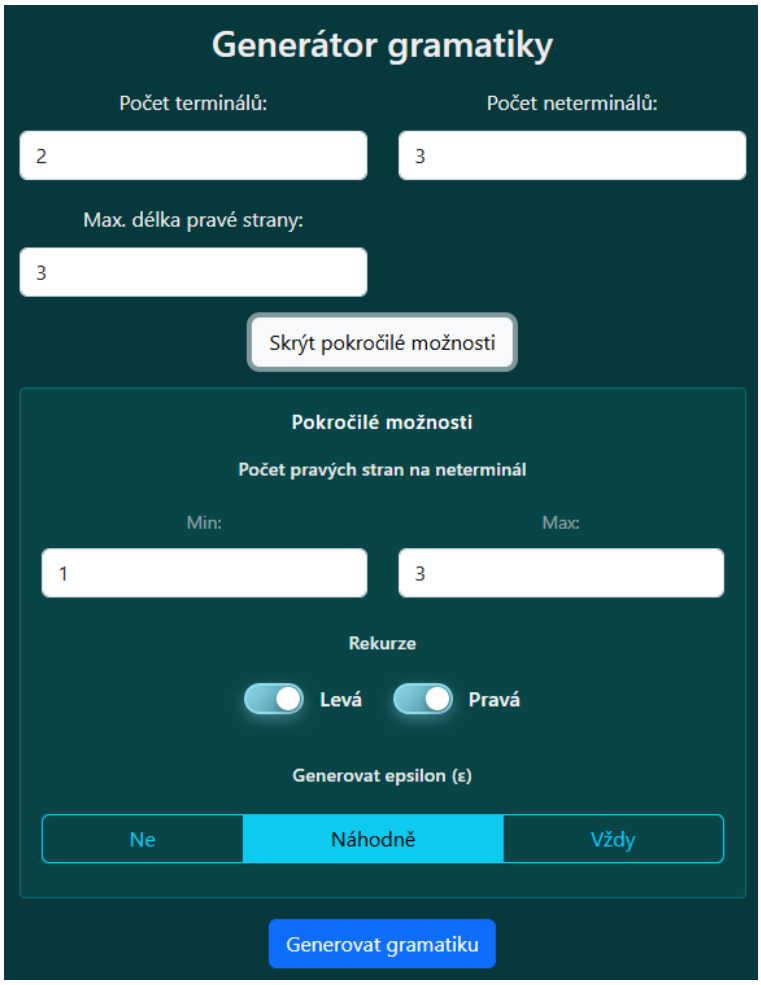
Pokročilé parametry umožňují jemnější kontrolu nad vlastnostmi generované gramatiky (viz obrázek 4.8):

- **Počet pravidel na neterminál:** Pole „Min“ a „Max“ určují rozsah, ve kterém se bude pohybovat počet pravidel pro každý neterminál. Pro obě pole je v uživatelském rozhraní povolen interval 1–8 a současně platí omezení, že minimální hodnota nesmí být větší než maximální. Výchozí hodnoty jsou 1 a 3.
- **Levá a pravá rekurze:** Přepínače „Levá“ a „Pravá“ umožňují vytvářet pravidla typu $A \to A\alpha$, respektive $A \to \alpha A$. Jsou-li zapnuty oba přepínače současně, pravidlo může obsahovat i centrální rekurzi typu $A \to A\alpha A$; její cílová pravděpodobnost je nastavena na 20%.
- **Režim epsilon pravidel:** Volba „Generovat epsilon (ε)“ nabízí tři možnosti:
  - „Ne“: Žádné epsilon pravidlo nebude vygenerováno.
  - „Náhodně“: Epsilon pravidla vznikají náhodně s malou pravděpodobností (8%).
  - „Vždy“: Algoritmus zaručí, že alespoň jedno epsilon pravidlo bude v gramatice přítomno.
- **Výkonové omezení konfigurace:** Před spuštěním generování aplikace odhaduje složitost zvolené konfigurace. U náročnějších nastavení uživatele upozorní na možné zpomalení a při příliš vysoké odhadované zátěži generování z výkonových důvodů neprovede.

Algoritmus generování

Algoritmus implementovaný ve funkci `generateGrammar()` prochází několika fázemi:

1. **Generování symbolů:**
   - Neterminály začínají velkým písmenem. První symbol je vždy `S` (startovní symbol), následují písmena `A`–`Z`, a při potřebě více symbolů pokračuje kombinacemi `AA`, `AB` atd.
   - Terminály začínají malým písmenem, konkrétně se používají písmena `a`–`z`.
2. **Vytvoření pravidel:** Pro každý neterminál se vygeneruje náhodný počet pravidel v rozsahu určeném poli „Min“ a „Max“.



Obrázek 4.8: Rozhraní pro nastavení parametrů generování bezkontextových gramatik

3. **Tvorba pravé strany pravidla:** Každé pravidlo vzniká následovně:
   - Podle režimu epsilon může být s určitou pravděpodobností vygenerováno prázdné pravidlo (ε).
   - Na základě přepínačů „Levá“ a „Pravá“ se rozhodne, zda bude pravidlo obsahovat rekurzivní volání aktuálního neterminálu na začátku nebo konci.
   - Pro zbývající část pravidla se náhodně vybírají symboly, přičemž s pravděpodobností 50% se vkládá terminál a s pravděpodobností 50% neterminál (kromě aktuálního, aby nedošlo k nechtěné přímé rekurzi mimo povolenou levou a pravou).
   - Maximální délka této nerekurzivní části odpovídá nastavené hodnotě pole „Max. délka pravé strany“ snížené o počet již přidaných rekurzivních symbolů.
4. **Zajištění dosažitelnosti:** Po vygenerování všech pravidel algoritmus provede průchod do šířky (BFS) ze startovního symbolu `S` a identifikuje všechny dosažitelné neterminály. Pro každý nedosažitelný neterminál následně náhodně vybere pravidlo některého dosažitelného neterminálu a upraví jej tak, aby odkazovalo na dosud nedosažitelný symbol. Tím je zaručeno, že všechny neterminály jsou využitelné v derivaci.
5. **Zajištění epsilon pravidla:** V režimu „Vždy“ algoritmus ověří, že alespoň jeden neterminál obsahuje epsilon pravidlo. Pokud ne, náhodně zvolí neterminál a doplní prázdné pravidlo tak, aby zůstal zachován nastavený horní limit počtu pravidel na neterminál (při dosaženém limitu jedno existující pravidlo nahradí, jinak nové pravidlo přidá).

Výsledná gramatika je tak validní bezkontextová gramatika s dosažitelnými neterminály a kontrolovanými vlastnostmi rekurze.

4.3.4 Předpřipravené sady

Ve složce `Sady/Grammar` jsou uloženy předpřipravené příklady gramatik ve formátu JSON (viz obrázek 4.9), jehož struktura je popsána v sekci 3.3.3.

4.3.5 Identifikace produktivních neterminálů

Modul `GrammarEvaluator.js` obsahuje implementaci algoritmu pro vyhodnocení problému neprázdnosti jazyka bezkontextové gramatiky založenou na teorii popsané v sekci 2.4. Algoritmus pracuje iterativně a postupně označuje produktivní neterminály. Implementace používá optimalizovanou verzi s frontou (work-list algorithm), což odpovídá principu BFS. Postup algoritmu:

1. **Inicializace:** Algoritmus vytvoří prázdnou množinu produktivních neterminálů a frontu pro zpracování. Všechna pravidla jsou zpočátku neoznačená.

```
1  {
2    "name": "Základní sčítání",
3    "nonTerminals": ["S", "E"],
4    "terminals": ["+", "id"],
5    "productions": {
6      "S": [
7        ["E"]
8      ],
9      "E": [
10       ["E", "+", "id"],
11       ["id"]
12     ]
13   }
14 }
```

Obrázek 4.9: Ukázka formátu JSON pro uložení bezkontextové gramatiky

2. **Nalezení základních produktivních pravidel:** Projdou se všechna pravidla gramatiky a identifikují se ta, jejichž pravá strana obsahuje pouze terminály nebo je prázdná (ε). Neterminály na levé straně těchto pravidel jsou okamžitě označeny jako produktivní a vloženy do fronty. Pokud nebylo nalezeno žádné takové pravidlo, fronta zůstane prázdná a další krok iterativní propagace se neprovede vůbec (přeskočí se krok 3.).

3. **Propagace produktivity:**

   - Algoritmus opakovaně prochází všechna pravidla gramatiky s dosud neoznačeným neterminálem na levé straně.
   - Pro každé takové pravidlo se zkontroluje, zda jsou všechny symboly na pravé straně buď terminály, nebo již označené produktivní neterminály.
   - Pokud ano, neterminál na levé straně tohoto pravidla je označen jako produktivní a dojde k nastavení příznaku změny na `true`.
   - Pokud ne (tedy pravá strana obsahuje alespoň jeden dosud neproduktivní neterminál), pravidlo zůstává nezměněno a bude znovu zkontrolováno v dalším průchodu.
   - Tento proces se opakuje, dokud se v posledním průchodu podařilo označit alespoň jeden nový neterminál jako produktivní, nebo dokud nejsou všechny neterminály gramatiky označeny jako produktivní.

4. **Ukončení:** Algoritmus končí, když je fronta prázdná. V tu chvíli jsou všechny produktivní neterminály nalezeny.

Časová složitost je $O(|P| \cdot |N|)$, kde $|P|$ je počet pravidel gramatiky a $|N|$ je počet neterminálů. V nejhorším případě je každé pravidlo kontrolováno jednou pro každý neterminál, který se stane produktivním.

4.3.6 Rekonstrukce derivace

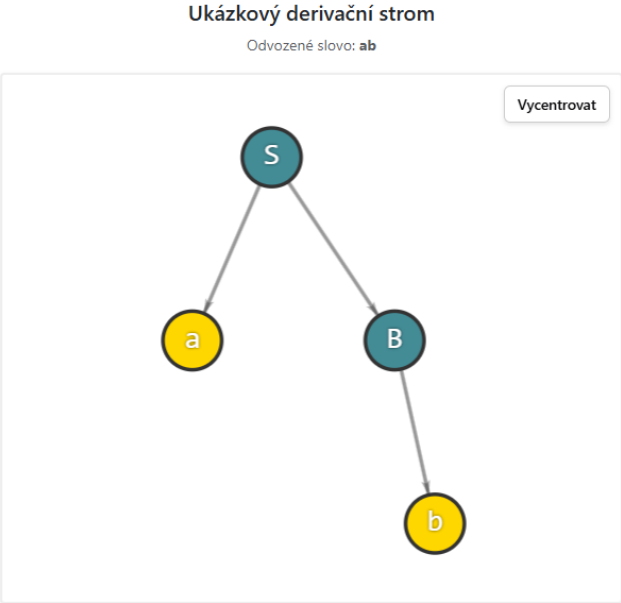
Pokud je počáteční symbol S produktivní, aplikace přistoupí k rekonstrukci ukázkové derivace. Nejprve se z množiny produktivních pravidel vypočítá pro každý neterminál odhad minimální délky odvoditelného terminálního řetězce. Tento krok je iterativní dopočet do ustálení (fixed-point výpočet): hodnoty se opakovaně zpřesňují, dokud se při dalším průchodu už žádná z nich nezmění.

Na základě vypočtených hodnot se pro každý produktivní neterminál vybere pravidlo s nejmenší cenou (a při shodě pravidlo s menším počtem symbolů na pravé straně). Teprve poté se vytvoří derivační strom a ukázkové slovo se získá přímo čtením terminálních listů zleva doprava.

Samotná konstrukce stromu je provedena iterativně pomocí zásobníku namísto hluboké rekurze, aby byla robustní i pro gramatiky s výraznou rekurzí. Použitý postup navazuje na standardní fixpoint/work-list přístup při analýze bezkontextových gramatik [9].

4.3.7 Vizualizace derivačního stromu

Pokud gramatika generuje neprázdný jazyk, aplikace vizuálně ukáže derivační strom pomocí komponenty `DerivationTreeVisual` (viz obrázek 4.10).

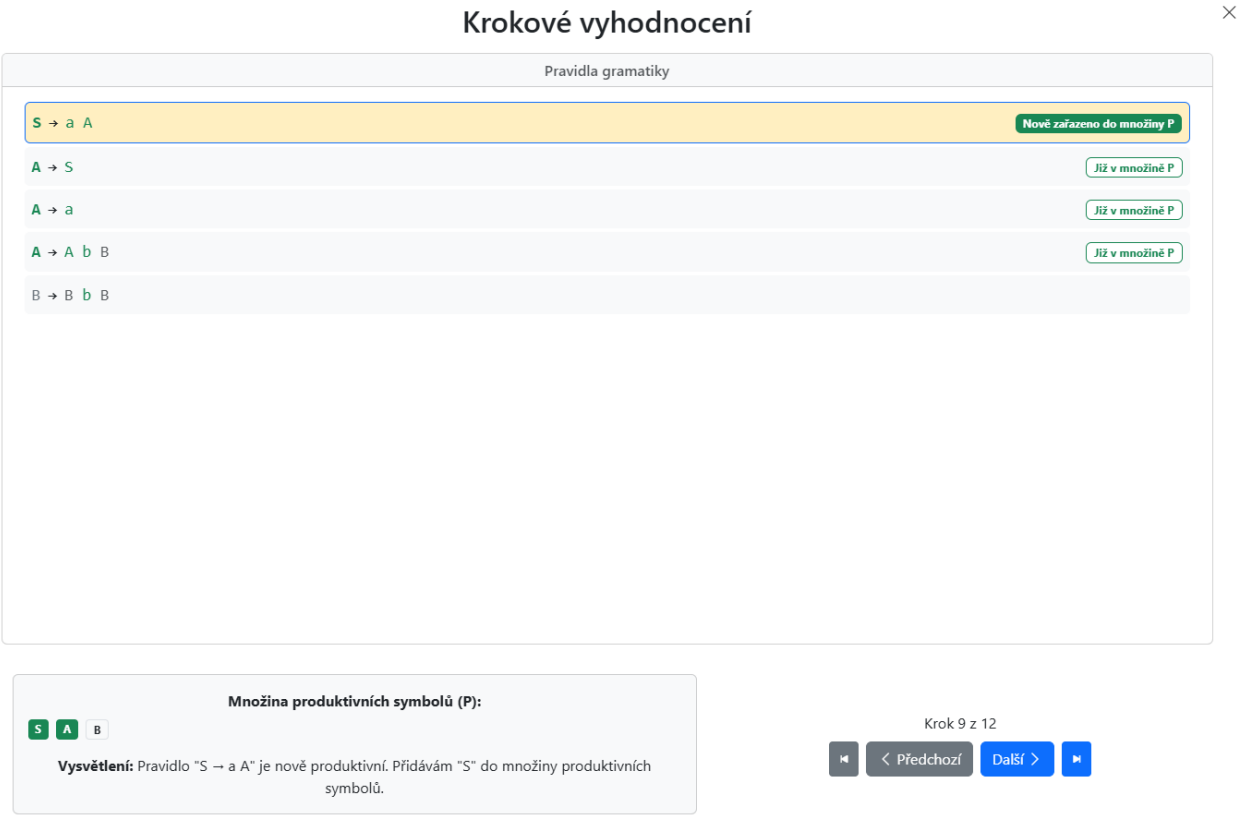


Obrázek 4.10: Vizualizace derivačního stromu pro neprázdnou gramatiku

4.3.8 Krokové vyhodnocení

Komponenta `GrammarStepEvaluator.js` vypočítá jednotlivé kroky pro komponentu `StepByStepGrammar`, která je následně interaktivně zobrazí. Zobrazuje seznam pravidel, přičemž barvou odlišuje jednotlivé symboly podle toho, zda jsou již považovány za produktivní. Aktuálně

posuzované pravidlo je zvýrazněno rámečkem a doplněno o stavový popisek (např. „Již v množině P“, „Nově zařazeno do množiny P“ nebo výčet chybějících symbolů), což pomáhá uživateli pochopit logiku algoritmu na úrovni jednotlivých kroků. Pod seznamem pravidel je zobrazena i množina všech dosud nalezených produktivních neterminálů (viz obrázek 4.11).



Obrázek 4.11: Kroková analýza bezkontextové gramatiky

4.4 Implementace převodů mezi problémy

Pro ilustraci vzájemné převoditelnosti P-úplných problémů popsané v sekci 2.5 aplikace implementuje převody z MCVP na ostatní dva problémy.

4.4.1 Převod na kombinatorickou hru

Třída `MCVPToGameStepGenerator` implementuje převod pomocí rekurzivního průchodu MCVP grafem. Pro každý uzel vytváří odpovídající pozici ve hře dle pravidel definovaných v sekci 2.5.1.

Algoritmus využívá několik klíčových mechanismů:

1. **Generování identifikátorů:** Každý uzel MCVP obvodu obdrží unikátní identifikátor ve formě `V0`, `V1`, `V2` atd. pomocí mapy `labelMap`, která zajišťuje konzistentní přiřazení – tentýž

uzel vždy obdrží stejný identifikátor. To je důležité pro správné propojení pozic v případě sdílených podgrafů.

2. **Sledování navštívených uzlů:** Množina `visited` zabraňuje duplicitnímu zpracování uzlů. Pokud algoritmus narazí na již navštívený uzel, rekurzivní průchod se okamžitě ukončí, protože odpovídající herní pozice již byla vytvořena. Tím je zajištěno, že každá pozice v herním grafu existuje právě jednou.

3. **Ponechání zpětných odkazů:** Pro každou pozici jsou kromě seznamu následovníků (`children`) udržovány i zpětné odkazy na předchůdce (`parents`). Tyto zpětné odkazy umožňují efektivní retrográdní analýzu, která propaguje výsledky směrem od terminálních pozic k počáteční pozici.

4. **Krokování převodu:** Pro vytvoření pozice a propojení pozic je zaznamenán krok obsahující popis akce, zvýrazněný uzel a aktuální stav grafu. Tyto kroky umožňují komponentě postupně vizualizovat celý proces převodu.

5. **Určení počáteční pozice:** Kořenový uzel MCVP obvodu se automaticky stává počáteční pozicí hry, která je ve vizualizaci označena oranžovou barvou.

Vizualizace probíhá pomocí komponenty `MCVPtoCombinatorialGameConverter`, která zobrazuje převod krokově (viz obrázek 4.12).

4.4.2 Převod na bezkontextovou gramatiku

Převod implementuje třída `MCVPToGrammarConverter` dle principů ze sekce 2.5.2. Algoritmus probíhá ve dvou fázích: nejprve přiřadí uzlům unikátní symboly a poté vytvoří odpovídající pravidla.

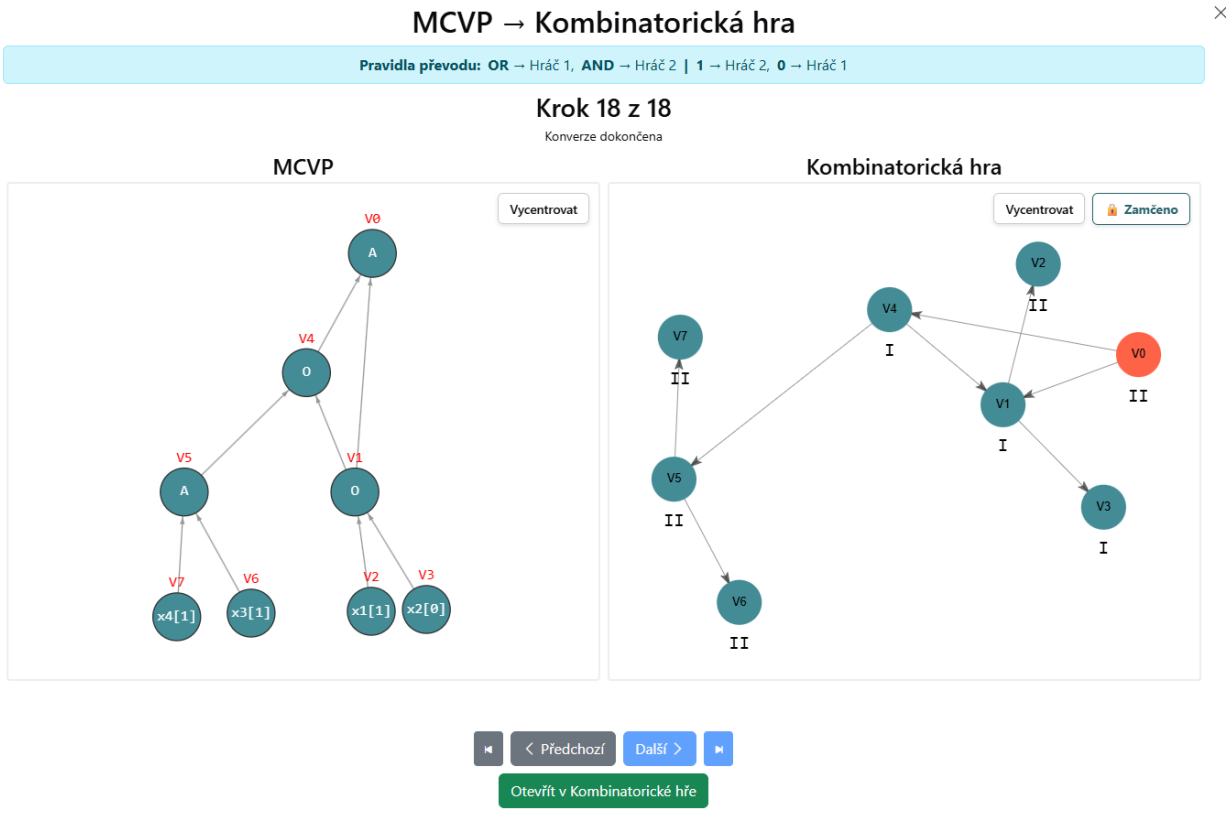
4.4.3 Generování symbolů gramatiky

Pro vytváření symbolů jsou použity dvě pomocné třídy:

- **NonTerminalGenerator:** Generuje neterminální symboly ve formě velkých písmen. Kořen obvodu vždy obdrží symbol `S` (startovní symbol gramatiky). Pro další uzly jsou použity symboly `A`–`Z`, a pokud je potřeba více symbolů, pokračuje se kombinacemi `AA`, `AB`, `AC` až `ZZ`.

- **TerminalGenerator:** Pro proměnné z MCVP obvodu s hodnotou 1 generuje jedinečný terminální symbol. Postupně jsou použity znaky `a`–`z`, následně speciální znaky (`+`, `-`, `=`, `*` atd.), a pokud je symbolů nedostatek, pokračuje se číslovanými terminály `t1`, `t2` atd. Je nastavena pravděpodobnost 15%, že proměnná s hodnotou 1 bude místo terminálu převedena na prázdné pravidlo (ε).

Uzly operací jsou reprezentovány neterminály. Proměnné s hodnotou 1 se převádějí na terminál (včetně možnosti ε) a proměnné s hodnotou 0 jsou reprezentovány neterminální smyčkou `X → X`.



Obrázek 4.12: Ukázka převodu MCVP obvodu na kombinatorickou hru

4.4.4 Tvorba pravidel

Druhá fáze algoritmu prochází orientovaným acyklickým grafem (DAG) reprezentujícím obvod a vytváří pravidla:

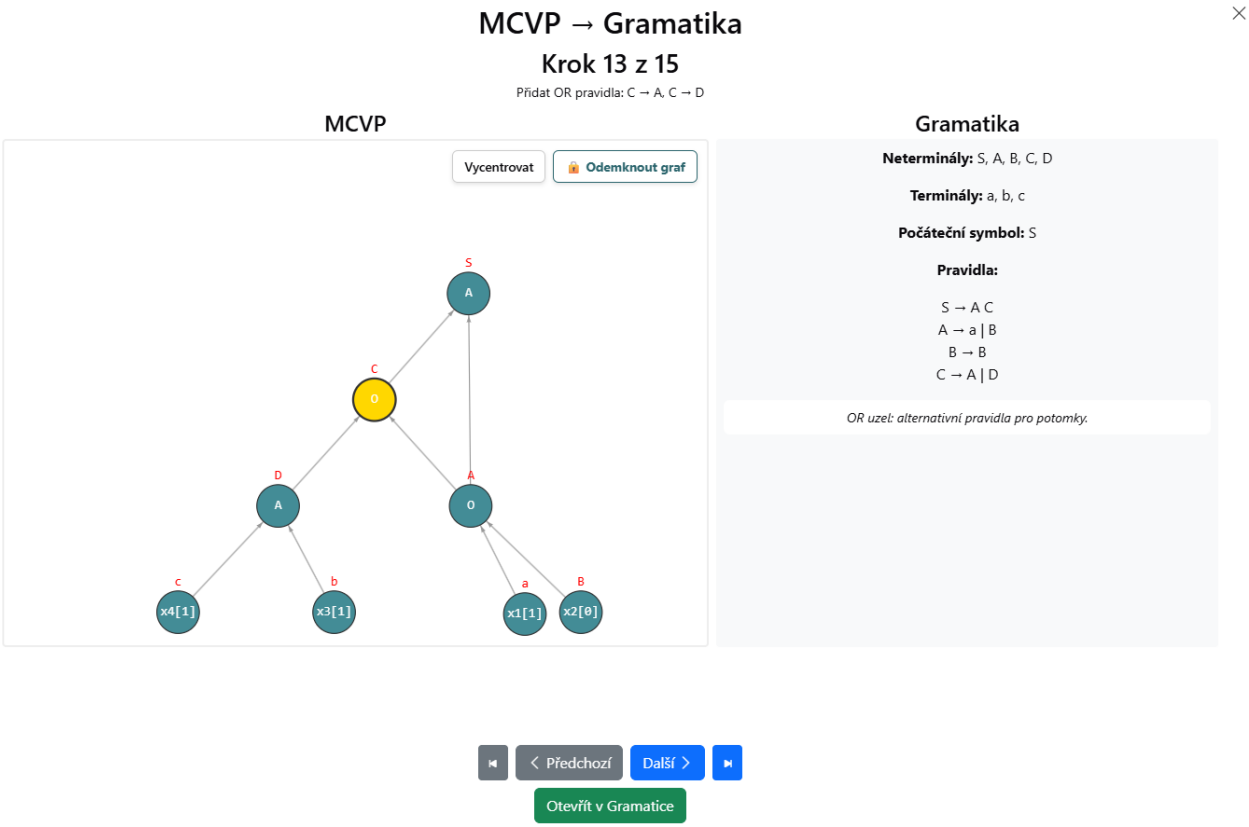
1. **Hradlo AND:** Vytvoří pravidlo ve formě `X → AB`, kde `X` je neterminál odpovídající hradlu a `A`, `B` jsou symboly odpovídající potomkům.

2. **Hradlo OR:** Vytvoří několik alternativních pravidel pro neterminál hradla, vždy jedno pro každého potomka: `X → A`, `X → B`, kde `A` a `B` jsou symboly odpovídající potomkům.

3. **Proměnná s hodnotou 1:** Nevytváří samostatný neterminál, ale je v pravidlech svých rodičů nahrazena přímo terminálem `a` (včetně možnosti ε s pravděpodobností 15 %). Pokud je tato proměnná kořenem obvodu, vznikne pravidlo `S → a` (případně `S →` ε).

4. **Proměnná s hodnotou 0:** Pro odpovídající neterminál `X` vznikne cyklické pravidlo `X → X`. Takový neterminál není produktivní, protože z něj nelze odvodit řetězec složený pouze z terminálů.

Výsledná gramatika přesně odpovídá hodnotě MCVP obvodu: gramatika generuje neprázdný jazyk právě tehdy, když obvod vyhodnotí hodnotu 1. Vizualizace probíhá pomocí komponenty `MCVPtoGrammarConverter`, která zobrazuje převod krokově (viz obrázek 4.13).



Obrázek 4.13: Ukázka převodu MCVP obvodu na bezkontextovou gramatiku

Kapitola 5

Závěr

Cílem této práce bylo vytvořit interaktivní webovou aplikaci pro vizualizaci a demonstraci převoditelnosti a řešení P-úplných problémů na příkladu tří vybraných P-úplných problémů: Monotone Circuit Value Problem, kombinatorických her na grafu a problému neprázdnosti jazyka bezkontextové gramatiky.

5.1 Dosažené výsledky

Výsledná aplikace splňuje všechny stanovené cíle a poskytuje jednotné prostředí pro práci se třemi P-úplnými problémy. Pro každý problém je implementováno kompletní řešení zahrnující:

- **Flexibilní vstupní způsoby:** Uživatelé mohou zadávat instance problémů třemi způsoby – manuálním zadáním, generováním náhodných instancí nebo pomocí předpřipravených příkladů.
- **Vizualizaci řešení:** Každý problém je doprovázen grafickou reprezentací, která využívá knihovnu *react-force-graph-2d* pro interaktivní zobrazení grafových struktur. Uživatel může manipulovat se zobrazenými grafy, přibližovat je a přesouvat uzly pro lepší orientaci.
- **Krokové vyhodnocení:** Implementace krokového průchodu algoritmů umožňuje sledovat každý jednotlivý krok výpočtu s textovým vysvětlením.
- **Export a import dat:** Možnost ukládání a načítání instancí problémů ve formátu JSON podporuje sdílení příkladů a vytváření připravených příkladů.

Aplikace také demonstruje dva konkrétní převody z MCVP na kombinatorickou hru a na bezkontextovou gramatiku. Oba převody jsou implementovány v krocích, takže uživatel může sledovat, jak se jednotlivé uzly obvodu transformují na odpovídající struktury v cílovém problému. Tato vizualizace ilustruje techniku polynomiálních redukcí a vzájemnou převoditelnost P-úplných problémů.

5.2 Vzdělávací přínos

Aplikace představuje vzdělávací nástroj pro výuku teorie složitosti s interaktivním přístupem. Krokové vyhodnocení s textovými vysvětleními umožňuje pochopit fungování algoritmů na konkrétních příkladech, zatímco vizualizace převodů demonstruje vzájemnou převoditelnost P-úplných problémů. Generování náhodných instancí pak podporuje experimentální pozorování chování algoritmů na různých instancích problémů.

5.3 Možnosti dalšího rozvoje

Přestože aplikace poskytuje funkční implementaci všech plánovaných funkcí, existuje prostor pro další rozšíření:

- **Další P-úplné problémy:** Aplikace by mohla být rozšířena o další P-úplné problémy, jako je například vyhodnocování booleovských formulí v konjunktivní normální formě nebo problém dosažitelnosti v grafech s omezenou šířkou.
- **Více převodů:** Implementace dalších směrů převodů – například z kombinatorických her na gramatiky nebo opačným směrem z gramatik na MCVP – by poskytla kompletnější obraz vzájemné převoditelnosti těchto problémů.

Aplikace je vhodná jako praktický doplněk výuky teorie složitosti, protože převádí abstraktní definice na konkrétní příklady.

Literatura

1. ARORA, Sanjeev; BARAK, Boaz. *Computational Complexity: A Modern Approach*. Cambridge University Press, 2009. ISBN 978-0521424264.
2. ASTURIANO, Vasco. *react-force-graph: React component for 2D, 3D, VR and AR force directed graphs [online]* [GitHub]. 2024. Dostupné také z: `https://github.com/vasturiano/react-force-graph`. [cit. 2026-02-03].
3. BOOTSTRAP TEAM. *Bootstrap: Build fast, responsive sites [online]* [OpenJS Foundation]. 2024. Dostupné také z: `https://getbootstrap.com/`. Verze 5.3.8. [cit. 2026-02-03].
4. BOSTOCK, Mike. *D3.js: Data-Driven Documents [online]* [Observable Inc.]. 2024. Dostupné také z: `https://d3js.org/`. Verze 7.9.0. [cit. 2026-02-03].
5. META PLATFORMS, INC. *React: The library for web and native user interfaces [online]* [Meta Open Source]. 2024. Dostupné také z: `https://react.dev/`. Verze 19.2. [cit. 2026-02-03].
6. MIYANO, Satoru; SHIRAISHI, Shunsuke; SHOUDAI, Takayoshi. *A List of P-Complete Problems [online]*. 1990-12. Tech. zpr., RIFIS-TR-CS-17. Fukuoka: Kyushu University, Research Institute of Fundamental Information Science. Dostupné také z: `https://catalog.lib.kyushu-u.ac.jp/opac_download_md/3123/rifis-tr-17.pdf`. [cit. 2026-02-03].
7. PAPADIMITRIOU, Christos H. *Computational Complexity*. Addison Wesley, 1994. ISBN 978-0201530827.
8. SAWA, Zbyněk. *Teoretická informatika – Podklady pro přednášky [online]* [Ostrava: VŠB-TUO, Katedra informatiky]. 2025. Dostupné také z: `https://www.cs.vsb.cz/sawa/ti/slides/ti-slides-08.pdf`. [cit. 2026-02-03].
9. SAWA, Zbyněk. *Úvod do teoretické informatiky – Bezkontextové gramatiky [online]* [Ostrava: VŠB-TUO, Katedra informatiky]. 2014. Dostupné také z: `https://www.cs.vsb.cz/kot/download/uti2014/uti-10-cz.pdf`. [cit. 2026-02-03].
10. SAWA, Zbyněk. *Úvod do teoretické informatiky – Turingovy stroje [online]* [Ostrava: VŠB-TUO, Katedra informatiky]. 2014. Dostupné také z: `https://www.cs.vsb.cz/kot/download/uti2014/uti-08-cz.pdf`. [cit. 2026-02-03].

11. SAWA, Zbyněk. *Úvod do teoretické informatiky – Výpočetní složitost algoritmů [online]* [Ostrava: VŠB-TUO, Katedra informatiky]. 2014. Dostupné také z: `https://www.cs.vsb.cz/kot/download/uti2014/uti-12-cz.pdf`. [cit. 2026-02-03].

12. VITE TEAM. *Vite: The Build Tool for the Web [online]* [VoidZero Inc.]. 2024. Dostupné také z: `https://vite.dev/`. Verze 6. [cit. 2026-02-03].

Příloha A

Obsah přiloženého archivu

Součástí této bakalářské práce je ZIP archiv obsahující textovou část práce ve formátu PDF a kompletní zdrojové kódy a sestavení aplikace.

Adresář / Soubor	**Popis**
`BachelorThesis.pdf`	Textová část bakalářské práce v digitální podobě.
`src/`	Zdrojové kódy aplikace v jazyce JavaScript (React).
`public/`	Statické soubory aplikace (ikony, manifest).
`Sady/`	Knihovna ukázkových instancí problémů ve formátu JSON.
`dist/`	Sestavená verze aplikace připravená k nasazení na webový server.
`package.json`	Konfigurační soubor Node.js obsahující seznam závislostí a skriptů.
`vite.config.js`	Konfigurace sestavovacího nástroje Vite.
`README.md`	Dokumentace se stručným návodem pro instalaci a spuštění.

Tabulka A.1: Seznam hlavních adresářů a souborů odevzdaného archivu

Příloha B

Technická příručka pro spuštění a nasazení

Tato příloha popisuje požadavky na prostředí a kroky potřebné pro lokální spuštění nebo produkční nasazení aplikace.

B.1 Systémové požadavky

Aplikace je implementována jako jednostránková webová aplikace (SPA) v rámci ekosystému Node.js. Pro práci se zdrojovými kódy je vyžadováno:

- **Node.js:** Doporučená verze 18.0.0 nebo novější.
- **Správce balíčků:** `npm` (součást instalace Node.js).
- **Moderní webový prohlížeč:** Podporující standardy ES6+ (Chrome, Firefox, Edge, Safari).

B.2 Instalace a lokální spuštění

Pro zprovoznění vývojového prostředí postupujte následovně:

1. Rozbalte obsah archivu do zvoleného adresáře.
2. V terminálu přejděte do kořenového adresáře aplikace.
3. Nainstalujte potřebné knihovny příkazem:
   `npm install`
4. Spusťte vývojový server příkazem:
   `npm run dev`
5. Aplikace bude dostupná na adrese `http://localhost:5173/`.

B.3 Produkční nasazení

Jelikož se jedná o čistě klientskou aplikaci bez vlastního backendu, lze ji hostovat na libovolném statickém webovém serveru.

1. Vygenerujte produkční sestavení příkazem:
   `npm run build`

2. Veškerý obsah vzniklého adresáře `dist/` nahrajte do kořenového adresáře vašeho webového serveru (např. Apache, Nginx) nebo využijte služby jako Vercel, Netlify či GitHub Pages.